

Course Code: CSE 499A

Section: 15

Project Group No: 06

Group Members:

Md.Murad Ul Momin (ID: 2232360642)

Rima Saha (ID: 2132517042)

Sakia Tabassum Anonna (ID: 2221433042)

Ashraful Hossain Riyan (ID: 2232845642)

PRIVATE SHIELD: Week 2 Report

I. INTRODUCTION

The PRIVATE SHIELD project continues the development of a Privacy-Preserving Federated Intelligence System for IoT Threat Detection. This report summarizes the recent milestones, focusing on active MQTT telemetry streaming, initial dataset construction, and the integration of advanced device behavior profiles. Additionally, we highlight the early completion of objectives regarding feature generation.

II. SYSTEM IMPLEMENTATION

Engineering efforts were focused on evolving the IoT simulator into a more robust, multi-profile traffic generator and establishing baseline data collection.

A. Advanced IoT Simulation

The `fake_device.py` script was significantly upgraded to support dynamic device behavior profiles, enabling distinct telemetry patterns for:

- Cameras: Port 554, high bandwidth (1400 bytes/pkt, 10/sec).
- Sensors: Port 1883, low frequency (32 bytes/pkt, 1/sec).
- Plugs: Port 80, bursty traffic (256 bytes/pkt, 0.2/sec).

Instances are now triggered via command-line arguments (e.g., `--type camera`), allowing simultaneous execution of heterogeneous traffic patterns.

B. Feature Engineering

Anticipating future requirements, the team has already completed the objective to align simulated features with N-BaIoT column structures. The simulator now injects complex feature metrics including flow duration, packet length means, and various TCP flag counts (FIN,

SYN, RST), enhancing the synthetic data realism to match authentic network captures.

C. Data Logging & Baseline Collection

The `mqtt_logger.py` utility was deployed to subscribe to the `devices/#` topic. This script parses incoming JSON payloads, appends precision timestamps, and tags data as 'normal' (label 0). The system successfully committed over 500 records to `data/normal.csv`, providing the foundation for anomaly training.

D. Attack Methodology Simulation

The `attack_simulator.py` framework now incorporates three modes: DoS floods (500 msg/sec), progressive port scanning (ports 1-1000), and synchronized Mirai botnet surges (50 concurrent connections). These modules allow for reliable testing of detection triggers under simulated threat conditions.

E. Status Monitoring

A lightweight terminal monitor (`monitor.py`) was implemented using native Python libraries. It provides real-time visibility into the system, displaying device counts, message velocity, and system health status by polling a shared counter file.

FUTURE OBJECTIVES

With tasks ahead of schedule, the team will focus on:

- Finalizing the `dataset_builder.py` to integrate simulated data with real N-BaIoT benchmarks.
- Implementing auto-labeling within `attack_simulator.py` for seamless labeling of DoS, port scan, and Mirai traffic.
- Building visualization scripts to validate class balance and feature distributions.